Files

Memory

Types of Memory

Registers, Cache Memory, Main Memory (Random Access Memory), Secondary Memory (Hard Disk)

Cache Memory

Small, fast memory. Located between CPU and main memory. Stores frequently accessed data. Levels: L1, L2, L3

Main Memory (RAM)

Volatile memory. Slower than cache memory. Stores data and instructions for CPU. Directly accessible by CPU.

Secondary Memory (Hard Disk)

Non-volatile storage. Large capacity. Slower access compared to main memory. Used for long-term storage.

Definition of a File

File

A named collection of related information that is recorded on secondary storage.

Components of a File

Data, Metadata.

Types of Files

Regular files, Directories, Special files (e.g., device files)

File Operations

Creation, Opening, Reading, Writing, Closing, Deleting.

File Attributes

Name, Type, Size, Location, Protection, Time of creation/modification/access.

File Access from Program

Introduction

- Programs often need to access files that are not automatically connected to standard input or output.
- The `cat` program illustrates this need by concatenating named files into standard output.
- To access a file, it must be opened using the `fopen` function, which returns a file pointer.
- The file pointer, of type `FILE*`, contains information about the file, and it is used for subsequent reads or writes.
- It points to a structure that contains information such as the location of a buffer, the current character position in the buffer, whether the file is being read or written, and whether errors or EOF have occurred.

`fopen` Function Signature

```
FILE *fopen(char *name, char *mode);
```

- Opens a file specified by `name` with the given `mode` (e.g., `"r"` for read, `"w"` for write, `"a"` for append).
- If system distinguishes between text and binary files, for the latter, a "b" must be appended to the mode string.
- Returns a file pointer of type `FILE*`.
- Opening an existing file for writing causes the old contents to be discarded, while opening for appending preserves them.
- If there's an error, `fopen` returns `NULL`.

`fclose` Function

```
int fclose(FILE *fp);
```

- Breaks the connection between the file pointer `fp` and the external file.
- Frees the file pointer for another file.
- Flushes the buffer in which `putc` is collecting output for an output file.

File Access Functions

- `int getc(FILE *fp)`: Reads the next character from the file.
- `int putc(int c, FILE *fp)`: Writes the character `c` to the file.
- `int getchar()`: Equivalent to `getc(stdin)`.
- `int putchar(int c)`: Equivalent to `putc(c, stdout)`.

Example: cat Program

```
#include <stdio.h>

int main(int argc, char *argv[]) {
  FILE *fp;
  void filecopy(FILE *ifp, FILE *ofp);

  if (argc == 1) filecopy(stdin, stdout);
  else
    while (--argc > 0)
      if ((fp = fopen(*++argv, "r")) == NULL)
          { printf("cat: can't open %s\n", *argv); return 1; }
      else
          { filecopy(fp, stdout); fclose(fp); }
  return 0;
}

void filecopy(FILE *ifp, FILE *ofp) {
  int c;
  while ((c = getc(ifp)) != EOF)
    putc(c, ofp);
}
```

Error Handling - Stderr and Exit

Introduction

- Error handling is essential for robust programs.
- The `cat` program's initial error handling is not ideal, especially when output is redirected.
- `stderr`, a second output stream, is introduced for error messages.
- The program is revised to print errors on `stderr`.

Revised cat Program

```
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char *argv[]) {
  FILE *fp; char *prog = argv[0]; /* program name for errors */
  void filecopy(FILE *ifp, FILE *ofp);
  if (argc == 1) {       /* no args; copy standard input */
    filecopy(stdin, stdout);
  } else {
    while (--argc > 0)
      if ((fp = fopen(*++argv, "r")) == NULL) {
        fprintf(stderr, "%s: can't open %s\n", prog, *argv);
        exit(1);
      } else {
        filecopy(fp, stdout); fclose(fp);
      }
  }
  if (ferror(stdout)) {
    fprintf(stderr, "%s: error writing stdout\n", prog); exit(2);
  }
  exit(0);
}
```

Error Handling Mechanisms

- `fprintf(stderr, ...)`: Sends error messages to `stderr` for immediate display.
- `exit(int status)`: Terminates program execution. Return values help identify success or failure.
- `ferror(FILE *fp)`: Checks if an error occurred on the specified file stream.
- `feof(FILE *fp)`: Checks if end-of-file has occurred on the specified file stream.

return, exit, abort

- return is used to exit from a function and return a value.
- exit() is used to terminate the entire program with a status code.
- abort() is used to terminate the program abruptly due to a critical error.

Error handling

- Redirecting output requires better error handling to prevent errors from being mixed with normal output.
- `stderr` provides a separate stream for error messages.
- The `exit` function terminates program execution with a return status.
- Functions like `ferror` and `feof` help in checking error conditions.
- Serious programs should handle errors sensibly and return meaningful status values.

Line Input and Output

Introduction

- The standard library provides routines for line input and output.
- The `fgets` function reads the next input line from a file, similar to `getline`.
- The `fputs` function writes a string to a file.
- `gets` and `puts` operate on `stdin` and `stdout`, respectively.
- Implementation details for `fgets` and `fputs` are shown from the standard library.
- `getline` can be implemented using `fgets`.

`fgets` Function

```
char *fgets(char *line, int maxline, FILE *fp)
```

- Reads the next input line (including newline) from file `fp` into `line`.
- At most `maxline-1` characters will be read, and the line is terminated with '\0'.
- Returns `line` normally; `NULL` on end of file or error.

`fputs` Function

```
int fputs(char *line, FILE *fp)
```

- Writes a string (without a newline) to file `fp`.
- Returns `EOF` if an error occurs, otherwise non-negative.

fgets and fputs Implementation

```
/* fgets: get at most n chars from iop */
char *fgets(char *s, int n, FILE *iop) {
  register int c;
  register char *cs;
  cs = s;
  while (--n > 0 && (c = getc(iop)) != EOF)
    if ((*cs++ = c) == '\n')
      break;
  *cs = '\0';
  return (c == EOF && cs == s) ? NULL : s;
}

/* fputs: put string s on file iop */
int fputs(char *s, FILE *iop) {
  int c;
  while (c = *s++)
    putc(c, iop);
  return ferror(iop) ? EOF : 0;
}
```

getline Function Implementation

```
/* getline: read a line, return length */
int getline(char *line, int max) {
  if (fgets(line, max, stdin) == NULL)
    return 0;
  else
    return strlen(line);
}
```

Input and Output in C

Standard Input and Output

Text Stream Model

- A text stream consists of a sequence of lines, each ending with a newline character.
- ANSI standard defines library functions that work on any system where C exists.
- Programs using standard library functions can be portable across systems.

Simple Input Mechanism

`getchar` Function

```
int getchar(void);
```

- Reads one character at a time from standard input (keyboard).
- Returns the next input character or `EOF` on end of file.
- `EOF` is typically -1, but tests should use `EOF` for portability; it is a symbolic constant defined in stdio.h

Input Redirection

Substitute File for Keyboard Input

```
prog <infile
```

- File `infile` can replace standard input (keyboard).
- "`<infile`" is not included in the command-line arguments in argv.
- Similar behavior with pipes for input from other programs:
- `otherprog | prog`
- pipes the standard output of otherprog into the standard input for prog.

Simple Output Mechanism

`putchar` Function

```
int putchar(int);
```

- Puts the character on standard output (default: screen).
- Returns the character written or `EOF` on error.
- Output can be directed to a file: `prog >outfile`.

Output with `printf`

Formatted Output

- `printf` sends formatted output to standard output.
- `putchar` and `printf` output can be interleaved.

Header Inclusion

```
#include <stdio.h>
```

- Every source file using I/O library functions must include `<stdio.h>`.
- Enables proper compilation and linkage.

Example: Lowercasing Program

lower.c

```
#include <stdio.h>
#include <ctype.h>

int main() {
    int c;
    while ((c = getchar()) != EOF)
        putchar(tolower(c));
    return 0;
}
```

Standard I/O

- Standard I/O functions facilitate interaction with programs.
- Use `getchar`, `putchar`, and `printf` for simple cases.
- `#include <stdio.h>` ensures correct library usage.
- Enclosure in $<$ and $>$ implies a search is made for the header in a standard set of places
- On UNIX systems, typically in the directory /usr/include

Formatted Output - `printf`

Introduction

- The `printf` function translates internal values to characters.
- It formats and prints arguments on the standard output under control of the format string.
- Returns the number of characters printed.

printf Function Signature

```
int printf(char *format, arg1, arg2, ...);
```

- `printf` converts, formats, and prints its arguments.
- `format` is the control string with conversion specifications.
- Returns the number of characters printed.

Format String Components

- Ordinary characters: copied to the output stream.
- Conversion specifications: begin with %, ends with a conversion character.
- Format: *%[flags][minwidth][.][precision] [length] character*
- Fields enclosed in [ ] show that they are optional